

Report: Tree Weight Propagation, DFS/BFS, and Sign-Flipping

Aibek Zhazykbek

Introduction

This report covers the analysis and implementation of weighted n -ary trees where each child receives weight $1/n$ of its parent. We construct trees of depth $N = 3$, implement Depth-First Search (DFS) and Breadth-First Search (BFS), verify their weight sums, and repeat the experiments with alternating sign flips. Both recursive and iterative versions of BFS are provided, followed by a justification why BFS should not be implemented recursively.

Problem 1: General Class for the Weighted Tree

Each node has:

- A numeric weight
- A list of children
- A parameter n (branching factor)

Since each child receives weight $W_{child} = \frac{1}{n}W_{parent}$, the structure is defined recursively.

Problem 2: Generate a Tree of Depth $N = 3$

The tree is constructed such that the root has weight 1, and each child receives weight $1/n$ of its parent.

Problem 3: Depth-First Recursive Sum

DFS recursively visits all nodes and returns the total sum of weights. The expected total must equal 1 for all n .

Problem 4: Breadth-First (Iterative) Sum

Using a queue, BFS also returns 1.

Problem 5: Sign-Flipping Variation

Each time a node is visited, its weight is multiplied by -1 . Expected results:

DFS sum = $+1$, then -1 ,

BFS sum = $+1$, then -1 .

Problem 6: Recursive and Non-Recursive BFS

Two full implementations are included:

- BFS (iterative): using a queue
- BFS (recursive): using recursion + queue argument

Problem 7: Why BFS Should Not Be Implemented Recursively?

Breadth-First Search expands nodes level by level. Recursion naturally follows a depth-based pattern, not breadth-based. Implementing BFS recursively is problematic because:

- It requires passing and mutating a queue argument, which breaks natural recursion flow.
- The call depth becomes equal to the number of nodes, potentially causing stack overflow.

- It becomes slower and less readable than the iterative version.

In contrast, the iterative queue-based BFS is optimal and safe.

Python Code

```
class Node:
    def __init__(self, weight, n):
        self.weight = weight
        self.children = []
        self.n = n

    def grow(self, depth):
        if depth == 0:
            return
        child_weight = self.weight / self.n
        for _ in range(self.n):
            child = Node(child_weight, self.n)
            self.children.append(child)
            child.grow(depth - 1)

# ----- DFS Recursive Weight Sum -----
def dfs_sum(node):
    total = node.weight
    for child in node.children:
        total += dfs_sum(child)
    return total

# ----- BFS Iterative Weight Sum -----
from collections import deque

def bfs_sum(node):
    q = deque([node])
    total = 0
    while q:
        current = q.popleft()
        total += current.weight
```

```

        for c in current.children:
            q.append(c)
    return total

# ----- DFS with Sign Flipping -----
def dfs_flip_sum(node):
    node.weight *= -1
    total = node.weight
    for child in node.children:
        total += dfs_flip_sum(child)
    return total

# ----- BFS with Sign Flipping -----
def bfs_flip_sum(node):
    q = deque([node])
    total = 0
    while q:
        current = q.popleft()
        current.weight *= -1
        total += current.weight
        for c in current.children:
            q.append(c)
    return total

# ----- BFS Recursive Version -----
def bfs_recursive(queue, total=0):
    if not queue:
        return total
    current = queue.popleft()
    total += current.weight
    for c in current.children:
        queue.append(c)
    return bfs_recursive(queue, total)

# ----- Build and Test Tree -----
def build_tree(n):

```

```

    root = Node(1, n)
    root.grow(3)
    return root

for n in [2, 3, 5, 10]:
    t = build_tree(n)
    print("n=", n)
    print("DFS:", dfs_sum(t))
    print("BFS:", bfs_sum(t))

    print("Flip_DFS:", dfs_flip_sum(t))
    print("Flip_BFS:", bfs_flip_sum(t))
    print()

```

Conclusion

We successfully implemented weighted n -ary trees and verified that both DFS and BFS correctly return total weight 1. Sign-flip experiments behave exactly as expected, and BFS recursive implementation was shown to be inferior to the iterative version.